

Normalizing Flow

Hyoung Joon, Kim

June 3, 2026

Contents

1	Introduction	2
2	Main Idea	2
2.1	Normalizing Flow	2
2.2	Invertible Layeres	2
2.2.1	Linear Flow	2
2.2.2	Elementwise Flow	2
2.2.3	Coupling Flow	3
2.2.4	Autoregressive Flow	3
3	Extensions of Normalizing Flow	4
3.1	Multi-scale Flow	4
3.2	Continuous Normalizing Flow	4

1 Introduction

Normalizing flow is a nonlinear latent variable model, where the likelihood can be evaluated without approximation, while sampling is still straightforward. The idea is to model a flexible probability distribution through an invertible neural network.

2 Main Idea

2.1 Normalizing Flow

The normalizing flow uses deep neural network to transform the probability distribution of latent variable to data distribution. Suppose $p_z(z)$ is a simple probability distribution, for example, a Gaussian distribution. To evaluate the likelihood of $\mathbf{x} = \mathbf{f}(\mathbf{z}, \mathbf{w})$, the distribution $p(\mathbf{x}|\mathbf{w})$, we can use change of variables formula. To use the change of variables formula, we require that $\mathbf{f}(\mathbf{z}, \mathbf{w})$ to be invertible about $\mathbf{z}$ for every value of $\mathbf{w}$. If we denote the inverse of $\mathbf{f}(\mathbf{z}, \mathbf{w})$ as $\mathbf{g}(\mathbf{z}, \mathbf{w})$, we get

$$p_{\mathbf{x}}(\mathbf{x}|\mathbf{w}) = p_{\mathbf{z}}(\mathbf{g}(\mathbf{x}, \mathbf{w})) \|D\mathbf{g}(\mathbf{x})\|$$

where $D\mathbf{g}(\mathbf{x})$ denotes the Jacobian of $\mathbf{g}$, and its elements are given by $D\mathbf{g}_{ij}(\mathbf{x}) = \frac{\partial g_i(\mathbf{x}, \mathbf{w})}{\partial x_j}$.

One consequence of forcing invertible network is that the dimension of $\mathbf{x}$ and $\mathbf{z}$ equals. Notice that invertible functions are still invertible when they are composed, so we can create invertible layers to create flexible, invertible network. There are several techniques for constructing an invertible neural network.

2.2 Invertible Layeres

The layers are mainly forward passes, from input data to latent variables.

2.2.1 Linear Flow

The simplest invertible network is **linear function**. But since it takes $\mathcal{O}(D^3)$ times to invert the $D \times D$ matrix, we parametrize it directly with LU decomposition;

$$\mathbf{W} = \mathbf{P}\mathbf{L}(\mathbf{U} + \mathbf{D})$$

where $\mathbf{P}$ is a fixed permutation matrix, $\mathbf{L}$ is a lower triangular matrix, $\mathbf{U}$ is a upper triangular matrix with zeros on the diagonal, and $\mathbf{D}$ is a diagonal matrix. Using this parametrization, $\mathbf{W}$ is invertible in $\mathcal{O}(D^2)$.

However notice that linear flow has poor expressive capability.

2.2.2 Elementwise Flow

The simplest nonlinear flow is **elementwise flow**. Elementwise flow applies a nonlinear invertible function pointwise;

$$\mathbf{f}(\mathbf{h}, \mathbf{w}) = \begin{bmatrix} f(h_1, \mathbf{w}) \\ \vdots \\ f(h_D, \mathbf{w}) \end{bmatrix}$$

where $f(\cdot, \mathbf{w})$ is a pointwise invertible function for all values of parameter $\mathbf{w}$. The Jacobian of elementwise flow is diagonal, so the determinant of it is just the product of diagonal entries.

However elementwise flow cannot express the correlation between features, since they do not mix features. So we use it with linear flow, or *coupling flow* which will be explained right after.

2.2.3 Coupling Flow

The **coupling flow** retain the invertability of a linear transformation while allowing flexibility. It divides the input $\mathbf{h}$ into two parts $\mathbf{h} = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \end{bmatrix}$ and create the flow.

Definition 2.2.1 (Coupling Flow)

Let $\mathbf{h} = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \end{bmatrix}$. Then, the coupling flow $\mathbf{f}(\mathbf{h}, \mathbf{w})$ with parameter $\mathbf{w}(\cdot)$ that takes $\mathbf{h}_1$ as input is defined as

$$\mathbf{f}(\mathbf{h}, \mathbf{w}) = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{g}(\mathbf{h}_2, \mathbf{w}(\mathbf{h}_1)) \end{bmatrix}$$

where $\mathbf{g}(\cdot, \mathbf{w})$ is an invertible layer for all values of $\mathbf{w}$.

The point is that $\mathbf{w}(\cdot)$ does not have to be invertible. It can be any kind of function, CNN, attention layer, or any other networks. The inversion of $\mathbf{f}(\mathbf{h}, \mathbf{w}) = \begin{bmatrix} \mathbf{h}_1' \\ \mathbf{h}_2' \end{bmatrix}$ is given as

$$\begin{bmatrix} \mathbf{h}_1' \\ \mathbf{g}^{-1}(\mathbf{h}_2', \mathbf{w}(\mathbf{h}_1')) \end{bmatrix}$$

Since the transform of second half of the parameters only depend on the first half of parameters, it does not represent sufficient correlation between features. We solve this problem by applying random permutation and shuffle the inputs, between layers.

2.2.4 Autoregressive Flow

Autoregressive flow is a generalization of coupling flow.

Definition 2.2.2 (Autoregressive Flow)

Let $\mathbf{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_D \end{bmatrix}$ be the input vector and let $\mathbf{h}_{a:b} = \begin{bmatrix} h_a \\ \vdots \\ h_b \end{bmatrix}$. Then, the **autoregressive flow** $\mathbf{f}(\mathbf{h}, \mathbf{w})$ is defined as

$$f_i(\mathbf{h}, \mathbf{w}) = g(h_i, \mathbf{w}(\mathbf{h}_{1:i-1}))$$

where *transformer* $g(\cdot, \mathbf{h})$ is an invertible function, and parameters $\mathbf{w}, \mathbf{w}(\mathbf{h}_1), \mathbf{w}(\mathbf{h}_1, \mathbf{h}_2) \dots$ are called *conditioners*.

The term transformer has nothing to do with the architecture transformer that uses attention mechanism. This network

can be evaluated parallel by masking the unused values, instead of really dividing the inputs and looping.

The inversion is quite inefficient since it cannot be done in parallel. Let $\begin{bmatrix} h'_1 \\ h'_2 \\ \vdots \\ h'_n \end{bmatrix} = \begin{bmatrix} g(h_1, \mathbf{w}) \\ g(h_2, \mathbf{w}(h_1)) \\ \vdots \\ g(h_n, \mathbf{w}(\mathbf{h}_{1:n-1})) \end{bmatrix}$. The the inversion is given as

$$\begin{aligned} h_1 &= g^{-1}(h'_1, \mathbf{w}) \\ h_2 &= g^{-1}(h'_2, \mathbf{w}(h_1)) \\ &\vdots \\ h_n &= g^{-1}(h'_n, \mathbf{w}(\mathbf{h}_{1:n-1})) \end{aligned}$$

Hence, the sampling is very slow. If we use an autoregressive flow that flows from latent space to input data space, the sampling would be fast, but the evaluation of likelihood will be slow this time. This is called as an **inverse autoregressive flow**. To enable both direction to be fast and efficient, is to first train masked autoregressive flow and use this to learn inverse autoregressive flow.

3 Extensions of Normalizing Flow

3.1 Multi-scale Flow

The constraint of the normalizing flow is that the dimensionality of latent space and data space must be equal. This causes very costly computations when high-dimensional dataset such as images are given. To resolve this, we use **multi-scale flows**, which partitions the input data and gradually fix the data, so that the computational cost decreases as the flow goes on.

In the sampling process, we pick a sample from deepest layer, and pass it through network until the variable which we fixed during training process is met. Then we pick a sample again from the distribution of the latent variable which we fixed during training process, and concatenate it to the original sample. We repeat this until we reach the input data layer.

3.2 Continuous Normalizing Flow

Instead of applying discrete network layers, we can increase the number of layers to ∞ and make it into continuous flow. The flow of initial latent distribution $\mathbf{z}(0) \sim p_0(\mathbf{z})$ to final data distribution $\mathbf{z}(T) = \mathbf{x} \sim p_T(\mathbf{x})$ is given as

$$\mathbf{z}(T) = \mathbf{z}(0) + \int_0^T \mathbf{f}(\mathbf{z}(t), \mathbf{w}) dt.$$

Then, the following holds due to the *instantaneous change of variables*

$$\frac{d \log p_t(\mathbf{z}(t))}{dt} = -\text{Tr} \left(\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{z}(t)} \right).$$

The log-likelihood is then given as

$$\log p_T(\mathbf{x}) = \log p_0(\mathbf{z}(0)) - \int_0^T \text{Tr} \left(\frac{d\mathbf{f}(\mathbf{z}(t), \mathbf{w})}{d\mathbf{z}(t)} \right) dt$$

As you can see, the main advantage of CNF is that the likelihood requires the trace of Jacobian instead of determinant. This is efficient, but we can make it even more efficient by using *Hutchinson's trace estimator*,

$$\text{Tr}(\mathbf{A}) = \mathbb{E}_{\epsilon}[\epsilon^\top \mathbf{A} \epsilon]$$

where ϵ is a random vector whose distribution has zero mean and unit covariance, and we approximate the estimator using a finite number of samples. Moreover, the network function $\mathbf{f}$ can consist of any network layer, which may not be invertible. The only constraint is that the dimension of input vector and output vectors must equal. It is possible to increase and decrease the dimension during the flow.

The CNF can be trained using the adjoint sensitivity method, like Neural ODE of residual networks. See [this document](#). However, this is still slow, and **flow matching** can be used to overcome this drawback.